

Report 2 – Vector Clock and Causality Analysis

Implemented rules

Local schedule: increment $V[i]$ and store a copy of the vector in the event record.

Send: increment $V[i]$ independently before each neighbor transmission and attach a copy to the message.

Receive: component-wise maximum of local and received vectors, then increment the receiver's own entry.

Final vectors

Node	Final vector
0	[74, 73, 73, 73, 73, 73, 73, 73, 73, 74]
1	[72, 74, 72, 69, 69, 69, 69, 69, 69, 70]
2	[72, 73, 74, 72, 69, 69, 69, 69, 69, 70]
3	[72, 73, 73, 74, 72, 69, 69, 69, 69, 70]
4	[72, 73, 73, 73, 74, 72, 69, 69, 69, 70]
5	[72, 73, 73, 73, 73, 74, 72, 69, 69, 70]
6	[72, 73, 73, 73, 73, 73, 74, 72, 69, 70]
7	[72, 73, 73, 73, 73, 73, 73, 74, 72, 70]
8	[72, 73, 73, 73, 73, 73, 73, 73, 74, 73]
9	[72, 73, 73, 73, 73, 73, 73, 73, 73, 74]

Ten event-pair classifications

A	B	Relation
N01-E01	N04-E01	CONCURRENT
N01-E01	N09-E01	CONCURRENT
N01-E01	N06-E01	CONCURRENT
N01-E01	N00-E01	BEFORE
N01-E01	N07-E01	CONCURRENT
N01-E01	N05-E01	CONCURRENT
N01-E01	N02-E01	BEFORE
N01-E01	N01-E02	BEFORE
N01-E01	N03-E01	BEFORE
N01-E01	N08-E01	CONCURRENT

Three concurrent cross-node pairs

A	B	Relation
N01-E01	N04-E01	CONCURRENT
N01-E01	N09-E01	CONCURRENT
N01-E01	N06-E01	CONCURRENT

Final calendar size and state hash

Node	Size	State hash
0	100	e6408fbb5797b2f2704dd547b06d6576263376320554423c1ee8de31ddc6dc03
1	100	e6408fbb5797b2f2704dd547b06d6576263376320554423c1ee8de31ddc6dc03
2	100	e6408fbb5797b2f2704dd547b06d6576263376320554423c1ee8de31ddc6dc03
3	100	e6408fbb5797b2f2704dd547b06d6576263376320554423c1ee8de31ddc6dc03
4	100	e6408fbb5797b2f2704dd547b06d6576263376320554423c1ee8de31ddc6dc03
5	100	e6408fbb5797b2f2704dd547b06d6576263376320554423c1ee8de31ddc6dc03
6	100	e6408fbb5797b2f2704dd547b06d6576263376320554423c1ee8de31ddc6dc03
7	100	e6408fbb5797b2f2704dd547b06d6576263376320554423c1ee8de31ddc6dc03
8	100	e6408fbb5797b2f2704dd547b06d6576263376320554423c1ee8de31ddc6dc03
9	100	e6408fbb5797b2f2704dd547b06d6576263376320554423c1ee8de31ddc6dc03

Analysis

Vector clocks can distinguish causal order from concurrency because BEFORE requires every vector component to be less than or equal to the other vector, with at least one strict inequality. If neither vector dominates the other, the events are concurrent.

All nodes may hold identical final calendars while ending with different logical clock values because message receives and sends update clocks even when they carry only duplicate state.

Compared with Lamport clocks, vector clocks retain per-node causal knowledge and therefore can identify concurrency; Lamport clocks provide an order consistent with causality but cannot by themselves prove two events are concurrent.